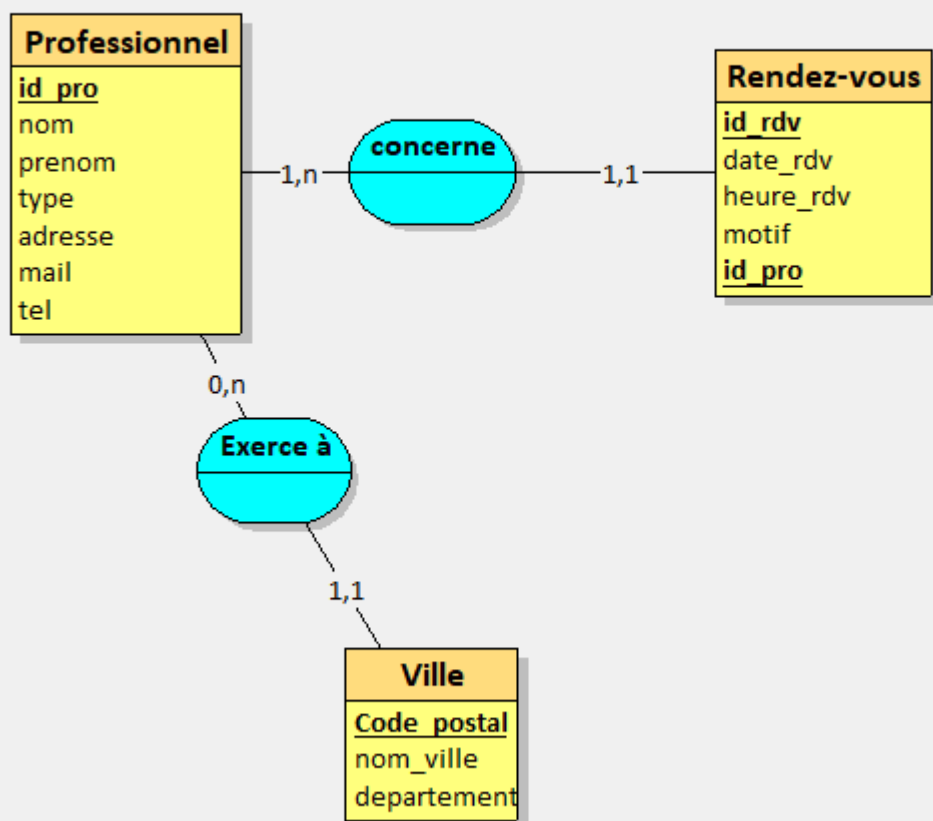


PROJET ANDROID

MCD :



MLD :

Professionnel (#id_pro, nom, prenom, type, adresse, mail, tel)

Ville (#Code_postal, nom_ville, departement)

Rendez-vous (#id_rdv, date_rdv, heure_rdv, motif, #id_pro)

CLASSE BD :

```
package com.example.projetandroid;

import android.content.ContentValues;
import android.content.Context;
import android.database.Cursor;
import android.database.sqlite.SQLiteDatabase;
import android.database.sqlite.SQLiteOpenHelper;

public class DatabaseHelper extends SQLiteOpenHelper {

    private static final String DATABASE_NAME = "gsb.db";
    private static final int DATABASE_VERSION = 1;

    // Table Professionnel
    public static final String TABLE_PRO = "professionnel";
    public static final String PRO_ID = "id_professionnel";
    public static final String PRO_NOM = "nom";
    public static final String PRO_PRENOM = "prenom";
    public static final String PRO_TYPE = "type";
    public static final String PRO_ADRESSE = "adresse";
    public static final String PRO_VILLE = "ville";
    public static final String PRO_CP = "code_postal";
    public static final String PRO_MAIL = "mail";
    public static final String PRO_TEL = "telephone";

    // Table RDV
    public static final String TABLE_RDV = "rdv";
```

```

public static final String RDV_ID = "id_rdv";
public static final String RDV_DATE = "date";
public static final String RDV_HEURE = "heure";
public static final String RDV_PRO_ID = "id_professionnel";

public DatabaseHelper(Context context) {
    super(context, DATABASE_NAME, null, DATABASE_VERSION);
}

@Override
public void onCreate(SQLiteDatabase db) {

    String createProfessionnel =
        "CREATE TABLE " + TABLE_PRO + " (" +
        PRO_ID + " INTEGER PRIMARY KEY AUTOINCREMENT, " +
        PRO_NOM + " TEXT, " +
        PRO_PRENOM + " TEXT, " +
        PRO_TYPE + " TEXT, " +
        PRO_ADRESSE + " TEXT, " +
        PRO_VILLE + " TEXT, " +
        PRO_CP + " TEXT, " +
        PRO_MAIL + " TEXT, " +
        PRO_TEL + " TEXT )";

    String createRdv =
        "CREATE TABLE " + TABLE_RDV + " (" +
        RDV_ID + " INTEGER PRIMARY KEY AUTOINCREMENT, " +
        RDV_DATE + " TEXT, " +
        RDV_HEURE + " TEXT, " +
        RDV_PRO_ID + " INTEGER, " +
        "FOREIGN KEY(" + RDV_PRO_ID + ") REFERENCES " +
        TABLE_PRO + "(" + PRO_ID + ") )";

    db.execSQL(createProfessionnel);
    db.execSQL(createRdv);
}

@Override
public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion) {
    db.execSQL("DROP TABLE IF EXISTS " + TABLE_RDV);
    db.execSQL("DROP TABLE IF EXISTS " + TABLE_PRO);
    onCreate(db);
}

public long insertProfessionnel(String nom, String prenom, String type,
                                String adresse, String ville, String cp,
                                String mail, String tel) {

    SQLiteDatabase db = this.getWritableDatabase();
    ContentValues values = new ContentValues();

    values.put(PRO_NOM, nom);
    values.put(PRO_PRENOM, prenom);
    values.put(PRO_TYPE, type);

```

```

        values.put(PRO_ADRESSE, adresse);
        values.put(PRO_VILLE, ville);
        values.put(PRO_CP, cp);
        values.put(PRO_MAIL, mail);
        values.put(PRO_TEL, tel);

        return db.insert(TABLE_PRO, null, values);
    }

    public long insertRdv(String date, String heure, int idProfessionnel) {
        SQLiteDatabase db = this.getWritableDatabase();
        ContentValues values = new ContentValues();

        values.put(RDV_DATE, date);
        values.put(RDV_HEURE, heure);
        values.put(RDV_PRO_ID, idProfessionnel);

        return db.insert(TABLE_RDV, null, values);
    }

    public Cursor getAllProfessionnels() {
        SQLiteDatabase db = this.getReadableDatabase();
        return db.rawQuery(
            "SELECT * FROM " + TABLE_PRO,
            null
        );
    }

    public Cursor getProfessionnelsByVilleOuCp(String recherche) {
        SQLiteDatabase db = this.getReadableDatabase();

        String query = "SELECT * FROM " + TABLE_PRO +
            " WHERE " + PRO_VILLE + " LIKE ? OR " + PRO_CP + " LIKE ?";

        String[] args = new String[]{
            "%" + recherche + "%",
            "%" + recherche + "%"
        };

        return db.rawQuery(query, args);
    }

    public Cursor getRdvByDate(String date) {
        SQLiteDatabase db = this.getReadableDatabase();

        String query = "SELECT r." + RDV_ID + ", r." + RDV_HEURE + ", " +
            "p." + PRO_NOM + ", p." + PRO_PRENOM +
            " FROM " + TABLE_RDV + " r INNER JOIN " + TABLE_PRO + " p " +
            "ON r." + RDV_PRO_ID + " = p." + PRO_ID +
            " WHERE r." + RDV_DATE + " = ? " +
            "ORDER BY r." + RDV_HEURE;

        return db.rawQuery(query, new String[]{date});
    }

```

```
}  
}
```

Activity_main.xml :

The image displays the visual layout and the corresponding XML structure for the Activity_main.xml file. The visual layout on the left shows a registration form with fields for Name, First Name, Type (pharmacist or Generalist), Email, Telephone, Address, City, and Postal Code. It includes a calendar for January 1970 and buttons for registration, validation, and viewing the calendar. The XML structure on the right lists the components used, such as TextViews for labels, EditTexts for input fields, RadioButtons for the type selection, a CalendarView, a Spinner, and Buttons for the actions.

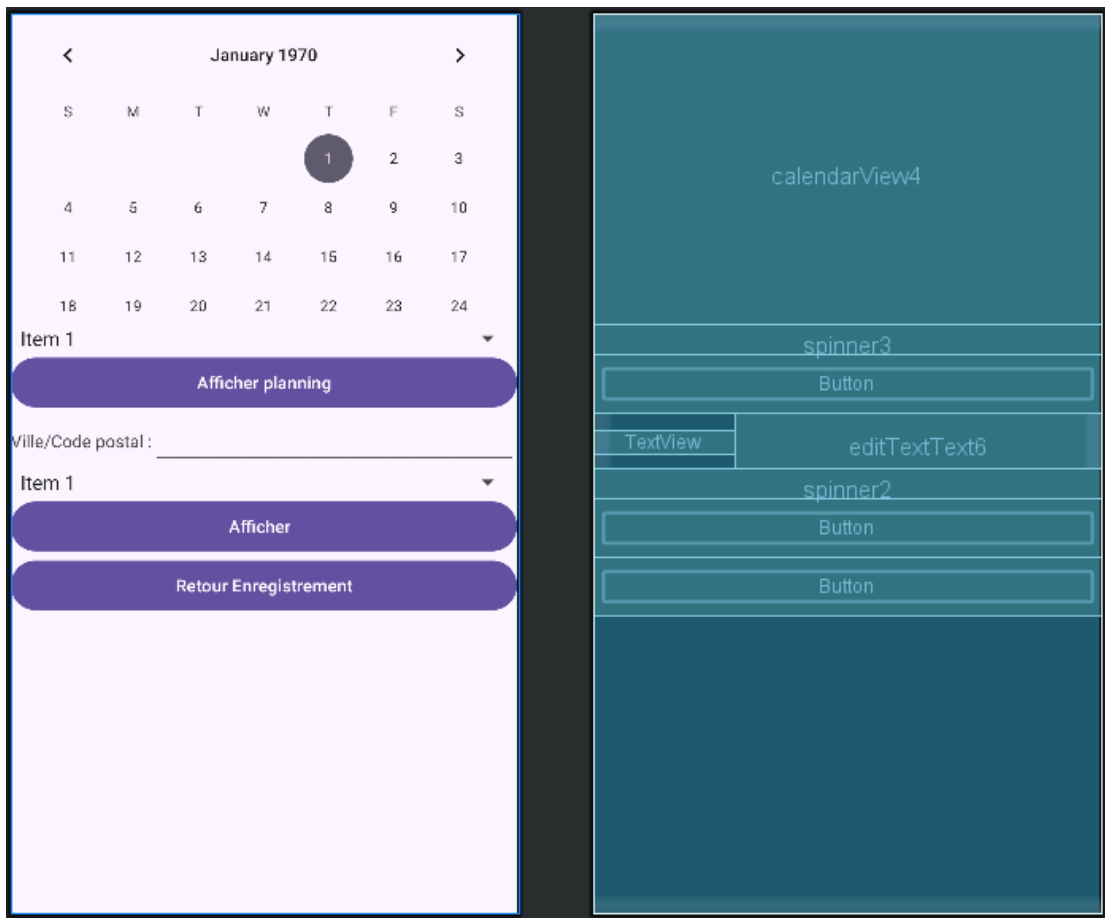
Visual Layout (Left):

- Nom :
- Prénom :
- Type : ☐ pharmacien ☐ Généraliste
- Mail :
- Téléphone :
- Adresse :
- Ville :
- Code Postal :
- Enregistrer Professionnel**
- Calendar: January 1970 (Day 1 is selected)
- Item 1
- Valider RDV**
- Voir le Planning / Recherche**

XML Structure (Right):

- TextView etNom
- TextView etPrenom
- TextView ☐ pharm ☐ Génér
- TextView etMail
- TextView etTelephone
- TextView etAdresse
- TextView etVille
- TextView etCodePostal
- Button
- calendarView
- spinner
- Button
- Button

Activity_main2.xml



MainActivity.java :

```
public class MainActivity extends AppCompatActivity {

    EditText etNom;
    EditText etPrenom;
    EditText etMail;
    EditText etTelephone;
    EditText etAdresse;
    EditText etVille;
    EditText etCodePostal;
    RadioGroup rgType;

    CalendarView calendarView;
    Spinner spinner;

    private String dateSelectionnee = "";

    private DatabaseHelper dbHelper;

    private List<Integer> listIds = new ArrayList<>();

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        EdgeToEdge.enable(this);
        setContentView(R.layout.activity_main);

        ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main),
(v, insets) -> {
            Insets systemBars =
insets.getInsets(WindowInsetsCompat.Type.systemBars());
            v.setPadding(systemBars.left, systemBars.top, systemBars.right,
systemBars.bottom);
            return insets;
        });

        etNom = findViewById(R.id.etNom);
        etPrenom = findViewById(R.id.etPrenom);
        etMail = findViewById(R.id.etMail);
        etTelephone = findViewById(R.id.etTelephone);
        etAdresse = findViewById(R.id.etAdresse);
        etVille = findViewById(R.id.etVille);
        etCodePostal = findViewById(R.id.etCodePostal);
        rgType = findViewById(R.id.rgType);

        calendarView = findViewById(R.id.calendarView);
        spinner = findViewById(R.id.spinner);
    }
}
```



```

        dbHelper = new DatabaseHelper(this);

        java.util.Calendar cal = java.util.Calendar.getInstance();
        dateSelectionnee = String.format("%02d/%02d/%04d",
            cal.get(java.util.Calendar.DAY_OF_MONTH),
            cal.get(java.util.Calendar.MONTH) + 1,
            cal.get(java.util.Calendar.YEAR));

        calendarView.setOnDateChangeListener((view, year, month, dayOfMonth)
-> {
            dateSelectionnee = String.format("%02d/%02d/%04d", dayOfMonth,
month + 1, year);
        });

        chargerProfessionnelsDansSpinner();
    }

    private void chargerProfessionnelsDansSpinner() {
        List<String> noms = new ArrayList<>();
        listIds.clear();

        android.database.Cursor cursor = dbHelper.getAllProfessionnels();
        if (cursor != null && cursor.moveToFirst()) {
            do {
                int id =
cursor.getInt(cursor.getColumnIndexOrThrow(DatabaseHelper.PRO_ID));
                String nom =
cursor.getString(cursor.getColumnIndexOrThrow(DatabaseHelper.PRO_NOM));
                String prenom =
cursor.getString(cursor.getColumnIndexOrThrow(DatabaseHelper.PRO_PRENOM));
                String type =
cursor.getString(cursor.getColumnIndexOrThrow(DatabaseHelper.PRO_TYPE));
                listIds.add(id);
                noms.add(prenom + " " + nom + " (" + type + ")");
            } while (cursor.moveToNext());
            cursor.close();
        }

        ArrayAdapter<String> adapter = new ArrayAdapter<>(
            this, android.R.layout.simple_spinner_item, noms);

        adapter.setDropDownViewResource(android.R.layout.simple_spinner_dropdown_ite
m);
        spinner.setAdapter(adapter);
    }

    public void EnregistrerClic(View v) {
        String nom = etNom.getText().toString().trim();
        String prenom = etPrenom.getText().toString().trim();
        String mail = etMail.getText().toString().trim();
        String telephone = etTelephone.getText().toString().trim();
    }

```

```

String adresse      = etAdresse.getText().toString().trim();
String ville        = etVille.getText().toString().trim();
String codePostal   = etCodePostal.getText().toString().trim();

int selectedId = rgType.getCheckedRadioButtonId();
String type = "Non défini";
if (selectedId != -1) {
    RadioButton rbSelected = findViewById(selectedId);
    type = rbSelected.getText().toString();
}

if (nom.isEmpty() || prenom.isEmpty()) {
    Toast.makeText(this, "Veuillez remplir le nom et le prénom",
Toast.LENGTH_SHORT).show();
    return;
}

long id = dbHelper.insertProfessionnel(
    nom, prenom, type, adresse, ville, codePostal, mail,
telephone);

if (id != -1) {
    Toast.makeText(this,
        "Professionnel enregistré : " + prenom + " " + nom,
        Toast.LENGTH_LONG).show();
    etNom.setText(""); etPrenom.setText(""); etMail.setText("");
    etTelephone.setText(""); etAdresse.setText("");
    etVille.setText(""); etCodePostal.setText("");
    rgType.clearCheck();
    chargerProfessionnelsDansSpinner();
} else {
    Toast.makeText(this, "Erreur lors de l'enregistrement",
Toast.LENGTH_SHORT).show();
}
}

public void ValiderRDVClic(View v) {
    if (listIds.isEmpty()) {
        Toast.makeText(this, "Aucun professionnel disponible",
Toast.LENGTH_SHORT).show();
        return;
    }

    if (dateSelectionnee.isEmpty()) {
        Toast.makeText(this, "Veuillez sélectionner une date",
Toast.LENGTH_SHORT).show();
        return;
    }

    int idPro = listIds.get(spinner.getSelectedItemId());

    long idRdv = dbHelper.insertRdv(dateSelectionnee, "00:00", idPro);

```

```
        if (idRdv != -1) {
            Toast.makeText(this,
                "RDV enregistré le " + dateSelectionnee,
                Toast.LENGTH_LONG).show();
        } else {
            Toast.makeText(this, "Erreur lors de l'enregistrement du RDV",
                Toast.LENGTH_SHORT).show();
        }
    }

    public void AllerVersPlanning(View v) {
        Intent intent = new Intent(this, MainActivity2.class);
        startActivity(intent);
    }
}
```

MainActivity2.java :

```
package com.example.projetoandroid;

import android.database.Cursor;
import android.os.Bundle;
import android.view.View;
import android.widget.AdapterView;
import android.widget.AdapterView.OnItemClickListener;
import android.widget.CalendarView;
import android.widget.EditText;
import android.widget.ListView;
import android.widget.Spinner;
import android.widget.Toast;

import androidx.activity.EdgeToEdge;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.graphics.Insets;
import androidx.core.view.ViewCompat;
import androidx.core.view.WindowInsetsCompat;

import java.util.ArrayList;
import java.util.List;

public class MainActivity2 extends AppCompatActivity {

    CalendarView calendarView4;
    Spinner spinner3;

    EditText editTextText6;
    Spinner spinner2;

    private String dateSelectionnee = "";

    private DatabaseHelper dbHelper;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        EdgeToEdge.enable(this);
        setContentView(R.layout.activity_main2);

        ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main),
(v, insets) -> {
            Insets systemBars =
insets.getInsets(WindowInsetsCompat.Type.systemBars());
            v.setPadding(systemBars.left, systemBars.top, systemBars.right,
systemBars.bottom);
            return insets;
        });
    }
}
```

```

        calendarView4 = findViewById(R.id.calendarView4);
        spinner3 = findViewById(R.id.spinner3);
        editTextText6 = findViewById(R.id.editTextText6);
        spinner2 = findViewById(R.id.spinner2);

        dbHelper = new DatabaseHelper(this);

        java.util.Calendar cal = java.util.Calendar.getInstance();
        dateSelectionnee = String.format("%02d/%02d/%04d",
            cal.get(java.util.Calendar.DAY_OF_MONTH),
            cal.get(java.util.Calendar.MONTH) + 1,
            cal.get(java.util.Calendar.YEAR));

        calendarView4.setOnDateChangeListener((view, year, month, dayOfMonth)
-> {
            dateSelectionnee = String.format("%02d/%02d/%04d", dayOfMonth,
month + 1, year);
        });
    }

    public void PlanningClic(View v) {
        List<String> rdvList = new ArrayList<>();

        Cursor cursor = dbHelper.getRdvByDate(dateSelectionnee);
        if (cursor != null && cursor.moveToFirst()) {
            do {
                String heure =
cursor.getString(cursor.getColumnIndexOrThrow(DatabaseHelper.RDV_HEURE));
                String nom =
cursor.getString(cursor.getColumnIndexOrThrow(DatabaseHelper.PRO_NOM));
                String prenom =
cursor.getString(cursor.getColumnIndexOrThrow(DatabaseHelper.PRO_PRENOM));
                rdvList.add(heure + " - " + prenom + " " + nom);
            } while (cursor.moveToNext());
            cursor.close();
        }

        if (rdvList.isEmpty()) {
            rdvList.add("Aucun RDV pour le " + dateSelectionnee);
        }

        ArrayAdapter<String> adapter = new ArrayAdapter<>(
            this, android.R.layout.simple_spinner_item, rdvList);

        adapter.setDropDownViewResource(android.R.layout.simple_spinner_dropdown_ite
m);

        spinner3.setAdapter(adapter);

        Toast.makeText(this, "Planning du " + dateSelectionnee,
Toast.LENGTH_SHORT).show();
    }

```

```

public void AfficheProfessionnel(View v) {
    String recherche = editTextText6.getText().toString().trim();

    if (recherche.isEmpty()) {
        Toast.makeText(this,
            "Veuillez saisir une ville ou un code postal",
            Toast.LENGTH_SHORT).show();
        return;
    }

    List<String> resultats = new ArrayList<>();
    Cursor cursor = dbHelper.getProfessionnelsByVilleOuCp(recherche);

    if (cursor != null && cursor.moveToFirst()) {
        do {
            String nom =
cursor.getString(cursor.getColumnIndexOrThrow(DatabaseHelper.PRO_NOM));
            String prenom =
cursor.getString(cursor.getColumnIndexOrThrow(DatabaseHelper.PRO_PRENOM));
            String type =
cursor.getString(cursor.getColumnIndexOrThrow(DatabaseHelper.PRO_TYPE));
            String ville =
cursor.getString(cursor.getColumnIndexOrThrow(DatabaseHelper.PRO_VILLE));
            String cp =
cursor.getString(cursor.getColumnIndexOrThrow(DatabaseHelper.PRO_CP));
            resultats.add(prenom + " " + nom + " - " + type + " (" + ville
+ " " + cp + ")");
        } while (cursor.moveToNext());
        cursor.close();
    }

    if (resultats.isEmpty()) {
        resultats.add("Aucun professionnel trouvé pour : " + recherche);
    }

    ArrayAdapter<String> adapter = new ArrayAdapter<>(
        this, android.R.layout.simple_spinner_item, resultats);

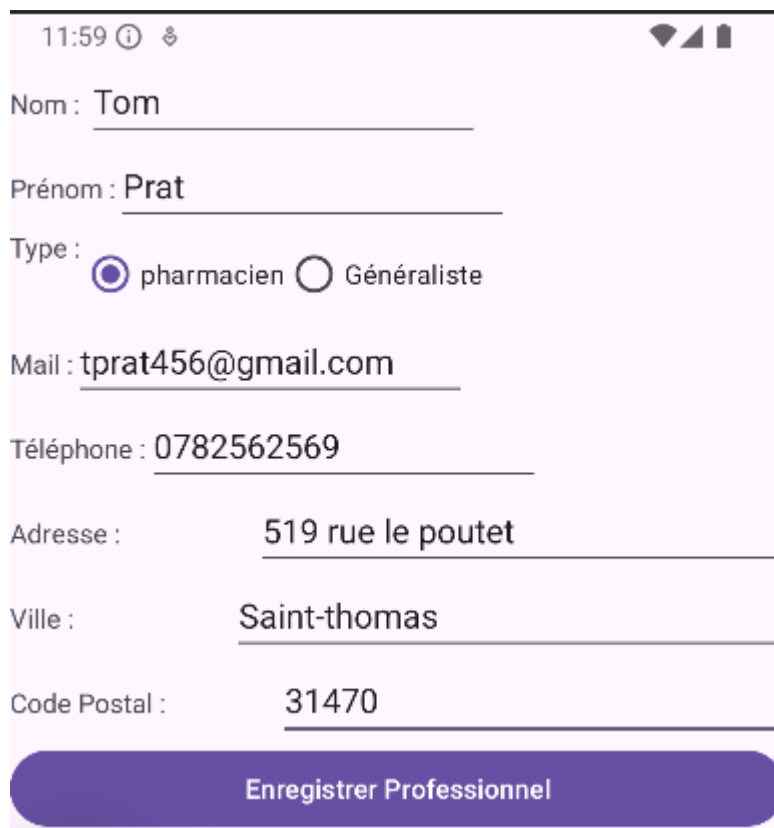
    adapter.setDropDownViewResource(android.R.layout.simple_spinner_dropdown_ite
m);
    spinner2.setAdapter(adapter);
}

public void RetourAccueil(View v) {
    finish();
}
}

```

Debug :

- Enregistrer un professionnel (nom, prénom, type : pharmacien, généraliste,..., adresse, mail, tél) :



The screenshot shows a mobile application interface for registering a professional. The form is set against a light purple background. At the top, the status bar shows the time 11:59 and various icons. The form fields are as follows:

- Nom : Tom
- Prénom : Prat
- Type : ☒ pharmacien ☐ Généraliste
- Mail : tprat456@gmail.com
- Téléphone : 0782562569
- Adresse : 519 rue le poutet
- Ville : Saint-thomas
- Code Postal : 31470

At the bottom of the form is a large, rounded purple button with the text "Enregistrer Professionnel" in white.

Une fois qu'on clique sur "Enregistrer Professionnel" le professionnel s'enregistre dans l'appli et on peut donc prendre un rdv avec ce professionnel.

- Prendre un RDV (date, heure, professionnel concerné) :

< May 2026 >

S	M	T	W	T	F	S
					1	2
3	4	5	6	7	8	9
10	11	12	13	14	15	16

Tom Prat (pharmacien) ▼

Valider RDV

- Afficher le planning d'une journée :

12:01 ⓘ 🔔

< May 2026 >

S	M	T	W	T	F	S
					1	2
3	4	5	6	7	8	9
10	11	12	13	14	15	16
17	18	19	20	21	22	23

00:00 — Tom Prat ▼

Afficher planning

Si je clique sur afficher le planning du 07/05/2026 on voit bien que j'y ai enregistré un rendez-vous avec mon professionnel.

- Afficher les professionnels avec recherche sur la ville ou le code postal.

Ville/Code postal : 31470

Tom Prat — pharmacien (Saint-Thomas 31470) ▼

 [Afficher](#)

Ici lorsque je rentre le code postal de mon professionnel il est directement trouvé dans la BD.